

			
1	2	3	4
8 npm install terser From NPM for programmatic use: npm install terser -g From NPM for use as a command line app: restart your computer after this step). latest version of node.js (You may need to First make sure you have installed the	7 uglify-es and uglify-js@3. retains API and CLI compatibility with uglify-js does not support ES6+. uglify-es is no longer maintained and Why choose terser? Find the changelog in CHANGELOG.md you use prettier . <i>Beautification</i> has been undocumented and is <i>being removed</i> from terser, we recommend	6 A JavaScript mangler/compressor toolkit for ES6+. <i>note: You can support this project on patreon: [link] The Terser Patron is shutting down in favor of opencollective.</i> Check out PATRONS.md for our first-tier patrons. Terser recommends you use RollupJS to bundle your modules, as that produces smaller code overall.	5
Install Command line usage terser [input files] [options] Terser can take multiple input files. It's recommended that you pass the input files first, then pass the options. Terser will parse input files in sequence and apply any compression options. The files are parsed in the same global scope, that is, a reference	from a file to some variable/function declared in another file will be matched properly. Command line arguments that take options (like -parse, -compress, -mangle and - format) can take in a comma-separated list of default option overrides. For instance: terser input.js --compress ecma=2015,computed_props=false If no input file is specified, Terser will read from STDIN.	If you wish to pass your options before the input files, separate the two with a double dash to prevent input files being used as option arguments: terser --compress --mangle -- input.js Command line options -h, --help Print usage information. -- help options` for details on available	options. -V, --version Print version number. -p, --parse <options> Specify parser options: `acorn` Use Acorn for parsing. `bare_returns` Allow return outside of functions. Useful when minifying CommonJS
6	10	11	12
16	15	14	13
with standard JavaScript globals and DOM API props. `debug` Add debug prefix and suffix. `keep-quoted` Only mangle unquoted properties, quoted properties are automatically reserved.	not used. -m, --mangle [options] Mangle names/specify mangler options: `reserved` List of names that should not be mangled. --mangle-props [options] Mangle properties/specify mangler options: `builtins` Mangle property names that overlaps	`SpiderMonkey` Assume input files are AST format (as JSON). -c, --compress [options] Enable compressor/specify compressor options: `pure_funcs` List of functions that can be safely removed when their return values are	modules and UserScripts that may be anonymous function wrapped (IIFE) by the .user.js engine `caller`. `expression` Parse a single expression, rather than a program (for parsing JSON).

`strict` disables quoted properties being automatically reserved. `regex` Only mangle matched property names. `only_annotated` Only mangle properties defined with / *@__MANGLE_PROP__*/. `reserved` List of names that should	not be mangled. -f, --format [options] Specify format options. `preamble` Preamble to prepend to the output. You can use this to insert a comment, for example for licensing information. This will not be parsed, but the	source map will adjust for its presence. `quote_style` Quote style: 0 - auto 1 - single 2 - double	3 - original `wrap_iife` Wrap IIFEs in parenthesis. Note: you may want to disable `negate_iife` under compressor options. `wrap_func_args` Wrap function arguments in parenthesis. -o, --output <file> Output
code removal or cascading statements into sequences. --config-file <file> Read 'minify()' options from JSON file. -d, --define <expr>=[value] Global definitions. --ecma <version> Specify ECMAScript release: 5, 2015, 2016, etc. -e, --enclose [arg:[value]] Embed	`false` to omit comments in the output -a valid JS RegExp like /foo/ or /i/ keep Note that currently not *all* comments can be kept when compression is on, because of dead	Closure, keeping JSDoc- style comments that contain e.g. "@license", or start with "!", You can optionally pass one of the following arguments to this flag: - "all" to keep all comments -	file path (default STDOUT). Specify 'ast' or 'spidermonkey' to write Terser or SpiderMonkey AST as JSON to STDOUT respectively. --comments [filter] Preserve copyright comments in the output. By default this works like Google
output in a big function with configurable arguments and values. --ie8 Support non-standard Internet Explorer 8. Equivalent to setting `ie8: true` in `minify()` for `compress`, `mangle` and `format`	options. By default Terser will not try to be IE-proof. --keep-classnames Do not mangle/drop class names. --keep-fnames Do not mangle/drop function names. Useful for code relying on Function.prototype.name. --module Input	is an ES6 module. If `compress` or `mangle` is enabled then the `toplevel` option, as well as strict mode, will be enabled. --name-cache <file> File to hold mangled name mappings. --safari10 Support non-standard Safari 10/11.	Equivalent to setting `safari10: true` in `minify()` for `mangle` and `format` options. By default `terser` will not work around Safari 10/11 bugs. --source-map [options] Enable source map/specify source map options: `base` Path to compute relative paths
top level scope. --wrap <name> Embed everything in a big function, making the "exports" and "global" variables available. You need to pass an argument to this option to specify the name that your module will take	be included in the source map. `url` If specified, path to the source map to append in `url` `//# sourceMappingURL`. --timings Display operations run time on STDERR. --toplevel Compress and/or mangle variables in	`filename` Name and/or location of the output source. `includeSources` Pass this flag if you want to include the content of source files in the source map as sourceContent property. `root` Path to the original source to	from input files. `content` Input source map, useful if you're compressing JS that was generated from some other original code. Specify "inline" if the source map is included within the sources.

[illegible]

<pre>\$ rm -f /tmp/cache.json # start fresh \$ terser file1.js file2.js --mangle-props --name-cache /tmp/cache.json -o part1.js \$ terser file3.js file4.js --mangle-props --name-cache /tmp/cache.json -o part2.js</pre> Now, part1.js and part2.js will be consistent with each other in terms of mangled property names.	Using the name cache is not necessary if you compress all your files in a single call to Terser. Mangling unquoted names (--mangle-props keep_quoted) Using quoted property name (o["foo"]) reserves the property name (foo) so that it is not mangled throughout the entire script even when used in an unquoted style (o.foo). Example:	<pre>// stuff.js var o = { "foo": 1, bar: 3 }; o.foo += o.bar; console.log(o.foo); \$ terser stuff.js --mangle-props keep_quoted -c -m var o={foo:1,o:3};o.foo+=o.o,console.log</pre>	Debugging property name mangling You can also pass --mangle-props debug in order to mangle property names without completely obscuring them. For example the property o.foo would mangle to o._\$foo\$_ with this option. This allows property mangling of a large codebase while still being able to debug the code and identify where mangling is breaking things.
49	50	51	52
56 Browser loading is also supported. It exposes a global variable Terser containing a .minify property: <pre><script src="https://cdn.jsdelivr.net/npm/source-map@0.7.3/dist/source-map.js"></script> <script src="https://cdn.jsdelivr.net/npm/terser/dist/bundle.min.js"></script></pre> There is an async high level function, async minify(code, options), which will	55 Assuming installation via NPM, you can load Terser in your application like this: <pre>const { minify } = require("terser"); import { minify } from "terser";</pre> Or,	54 Input script with new properties), and to help identify mistakes like writing mangled keys to with different inputs (e.g. as you update the Terser stuff.js --mangle-props debug -c -m	53 You can also pass a custom suffix using --mangle-props debug=XYZ. This would then mangle o.foo to o._\$foo\$XYZ. You can change this each time you compile a script to identify how a property got mangled. One technique is to pass a random number on every compile to simulate mangling changing <pre>var o={_\$foo\$_:1,\$bar\$_:3};o._\$foo\$_+=o</pre>
perform all minification phases in a configurable manner. By default minify() will enable compress and mangle. Example: <pre>var code = "function add(first, second) { return first + second; }"; var result = await minify(code, { sourceMap: true });</pre>	<pre>console.log(result.code); // minified output: function add(n,d) {return n+d} console.log(result.map); // source map</pre> There is also a minify_sync() alternative version of it, which returns instantly. You can minify more than one JavaScript file at a time by using an object for the first argument where the keys are file names and the values are source code:	<pre>var code = { "file1.js": "function add(first, second) { return first + second; }", "file2.js": "console.log(add(1 + 2, 3 + 4));" }; var result = await minify(code); console.log(result.code); // function add(d,n){return d+n} console.log(add(3,7));</pre> The toplevel option:	<pre>var code = { "file1.js": "function add(first, second) { return first + second; }", "file2.js": "console.log(add(1 + 2, 3 + 4));" }; var options = { toplevel: true }; var result = await minify(code, options); console.log(result.code); // console.log(3+7);</pre>
57	58	59	60
64 <pre>var cacheFileName = "/tmp/cache.json"; var options = { mangle: { properties: true, }, nameCache: JSON.parse(fs.readFileSync(cacheFileName, "utf8")) }; fs.writeFileSync("part1.js", await minify({ "file1.js": fs.readFileSync("file1.js", "utf8"), "file2.js": fs.readFileSync("file2.js", "utf8"), "file3.js": fs.readFileSync("file3.js", "utf8"), "file4.js": fs.readFileSync("file4.js", "utf8") } , options, "utf8"));</pre>	63 <pre>var cacheFileName = "/tmp/cache.json"; var options = { mangle: { properties: true, }, nameCache: JSON.parse(fs.readFileSync(cacheFileName, "utf8")) }; fs.writeFileSync("part1.js", await minify({ "file1.js": fs.readFileSync("file1.js", "utf8"), "file2.js": fs.readFileSync("file2.js", "utf8"), "file3.js": fs.readFileSync("file3.js", "utf8"), "file4.js": fs.readFileSync("file4.js", "utf8") } , options, "utf8"));</pre>	62 You may persist the name cache to the file system in the following way: <pre>var result2 = await minify({ "file2.js": "console.log(add(1 + 2, 3 + 4));" }, options); console.log(result2.code); // function n(n,r){return n+r} console.log(result2.code); // console.log(n(3,7));</pre>	61 <pre>The nameCache option: var options = { mangle: { toplevel: true, }, nameCache: {} }; var result1 = await minify({ "file1.js": "function add(first, second) { return first + second; }" }, options);</pre>

<pre> "file4.js": fs.readFileSync("file4.js", "utf8") }, options).code, "utf8"); fs.writeFileSync(cacheFileName, JSON.stringify(options.nameCache), "utf8"); </pre> An example of a combination of minify() options:	<pre> var code = { "file1.js": "function add(first, second) { return first + second; }", "file2.js": "console.log(add(1 + 2, 3 + 4));" }; var options = { toplevel: true, compress: { global_defs: { "@console.log": "alert" } } } </pre>	<pre> }, passes: 2 }, format: { preamble: "/* minified */" } }; var result = await minify(code, options); console.log(result.code); // /* minified */ // alert(10);" </pre>	An error example: <pre> try { const result = await minify({"foo.js" : "if (0) else console.log(1);"}); // Do something with result } catch (error) { const { message, filename, line, col, pos } = error; // Do something with error } </pre>
65	66	67	68
72 cache state of this object and update it during write property.minify(). will read the name invocations of minify(). Note: this is a read/and property names across multiple object if you wish to cache mangled variable object {} or a previously used nameCache variable and functions. - nameCache (default null) - pass an empty function name mangling and to drop unused you wish to enable top level variable and minifying an ES6 module, "use strict" is implied and names can be mangled on the top scope. If compress or mangle is enabled then the toplevel option will be enabled. - format or output (default null) — pass an object if you wish to specify additional format options. The defaults are optimized for best compression. - sourceMap (default false) - pass an object if you wish to specify source map options.	71 - module (default false) — Use when minifying an ES6 module, "use strict" is implied and names can be mangled on the top scope. If compress or mangle is enabled then the toplevel option will be enabled. - format or output (default null) — pass an object if you wish to specify additional format options. The defaults are optimized for best compression. - sourceMap (default false) - pass an object if you wish to specify source map options.	70 - parse (default {}) — pass an object if you wish to specify some additional parse options. - compress (default {}) — pass false to skip compressing entirely. Pass an object to specify custom compress options. - mangle (default true) — pass false to skip mangling names, or pass an object to specify mangle options (see below). - mangle.properties (default false) — a subcategory of the mangle option. Pass an object to specify custom mangle property options.	69 - ecma (default undefined) - pass 5, 2015, 2016, etc to override compress and format's ecma options. - enclose (default false) - pass true, or a string in the format of "args:values", where args and values are comma-separated argument names and values, respectively, to embed the output in a big function with the configurable arguments and values.
73 minification so that it may be reused or externally persisted by the user. - ie8 (default false) - set to true to support IE8. - keep_classnames (default: undefined) - pass true to prevent discarding or mangling of class names. Pass a regular expression to only keep class names matching that regex. - keep_fnames (default: false) - pass true to prevent discarding or mangling of function names. Pass a regular expression to only keep function names matching that regex.	74 Useful for code relying on Function.prototype.name. If the top level minify option keep_classnames is undefined it will be overridden with the value of the top level minify option keep_fnames. safari10 (default: false) - pass true to work around Safari 10/11 bugs in loop scoping and await. See safari10 options in mangle and format for details.	75 Minify options structure <pre> { parse: { // parse options }, compress: { // compress options }, mangle: { // mangle options properties: { </pre>	76 <pre> // mangle property options }, format: { // format options (can also use 'output' for backwards compatibility) }, sourceMap: { // source map options }, </pre>
80 <pre> var result = await minify({"file1.js": "var a = function() { {}, { sourceMap: { </pre> be included in source map: You can also specify sourceRoot property to "inline" and source map will be appended to code. You can set option sourceMap.url to be source map file. used to set file attribute (see the spec) in	79 <pre> console.log(result.code); // minified output console.log(result.map); // source map </pre> Note that the source map is not saved in a file, it's just returned in result.map. The value passed for sourceMap.url is only used to set sourceMap.url=sourceMap.url in result.code. The value of filename is only	78 <pre> var result = await minify({"file1.js": "var a = function() { {}, { sourceMap: { filename: "out.js", url: "out.js.map" } } }); </pre> To generate a source map: Source map options <pre> toplevel: false } </pre>	77 <pre> ecma: 5, // specify one of: 5, 2015, 2016, etc. enclose: false, // or specify true, or "args:values" keep_classnames: false, keep_fnames: false, ie8: false, module: false, nameCache: null, // or specify a name cache object safari10: false, </pre>

<pre> root: "http://example.com/ src", url: "out.js.map" } }); </pre> If you're compressing compiled JavaScript and have a source map for it, you can use <code>sourceMap.content</code>:	<pre> var result = await minify({"compiled.js": "compiled code"}, { sourceMap: { content: "content from compiled.js.map", url: "minified.js.map" } }); // same as before, it returns `code` and `map` </pre>	If you're using the X-SourceMap header instead, you can just omit <code>sourceMap.url</code>. If you happen to need the source map as a raw object, set <code>sourceMap.asObject</code> to <code>true</code>. Parse options <code>bare_returns</code> (default <code>false</code>) – support top level return statements <code>html5_comments</code> (default <code>true</code>) <code>shebang</code> (default <code>true</code>) – support <code>#!</code> command as the first line	<code>spidermonkey</code> (default <code>false</code>) – accept a Spidermonkey (Mozilla) AST Compress options <code>defaults</code> (default: <code>true</code>) – Pass <code>false</code> to disable most default enabled compress transforms. Useful when you only want to enable a few compress options while disabling the rest. <code>arrows</code> (default: <code>true</code>) – Class and object literal methods are converted will also be converted to arrow expressions if the
81 88 <code>drop_console</code> (default: <code>false</code>) – Pass <code>true</code> to discard calls to <code>console.*</code> functions. If you only want to discard a portion of console, you can pass an array like this <code>['log', 'info']</code>, which will only discard <code>console.log</code>. <code>drop_debugger</code> (default: <code>true</code>) – remove <code>debugger</code> statements <code>ecma</code> (default: 5) – Pass 2015 or greater to enable compress options that will transform ES5 code into smaller ES6+ equivalent forms.	82 87 <code>computed_props</code> (default: <code>true</code>) – Transforms constant computed properties into regular ones: <code>{["computed"]: 1}</code> is converted to <code>{computed: 1}</code>. <code>conditionals</code> (default: <code>true</code>) – apply optimizations for <code>if</code>-s and conditional expressions <code>dead_code</code> (default: <code>true</code>) – remove unreachable code <code>directives</code> (default: <code>true</code>) – remove redundant or non-standard directives	83 86 <code>collapse_vars</code> (default: <code>true</code>) – Collapse <code>instead of === and !==</code>. <code>single_use_non_constant_variables</code>, side effects permitting. <code>comparisons</code> (default: <code>true</code>) – apply certain optimizations to binary nodes, e.g. <code>!(a < b) -> a >= b</code> (only when <code>unsafe_comps</code>), attempts to negate binary nodes, e.g. <code>a == b && !c && !d && !e -> a != b c d e</code> etc. Note: comparisons as <code>i(b) c d e</code>) works best with <code>lhs_constants</code> enabled.	84 85 <code>resultant code is shorter: m() {return x} becomes m: () => x.</code> To do this to regular ES5 functions which don't use <code>this</code> or arguments, see <code>unsafe_arrows</code>. <code>arguments</code> (default: <code>false</code>) – replace <code>arguments[index]</code> with function parameter name whenever possible. <code>booleans</code> (default: <code>true</code>) – various optimizations for boolean context, for example <code>!!a ? b : c -> a ? b : c</code> <code>booleans_as_integers</code> (default: <code>false</code>) – Turn booleans into 0 and 1, also makes
89 96 <code>evaluate</code> (default: <code>true</code>) – attempt to evaluate constant expressions <code>expression</code> (default: <code>false</code>) – Pass <code>true</code> to preserve completion values from terminal statements without return, e.g. in bookmarklets. <code>global_defs</code> (default: <code>{}</code>) – see conditional compilation <code>hoist_funs</code> (default: <code>false</code>) – hoist function declarations <code>hoist_props</code> (default: <code>true</code>) – hoist properties from constant object and array	90 95 literals into regular variables subject to a set of constraints. For example: <code>var o={p:1, q:2}; f(o.p, o.q);</code> is converted to <code>f(1, 2);</code>. Note: <code>hoist_props</code> works best with <code>mangle</code> enabled, the <code>compress</code> option passes set to 2 or higher, and the <code>compress</code> option <code>toplevel</code> enabled. <code>hoist_vars</code> (default: <code>false</code>) – hoist var declarations (this is <code>false</code> by default because it seems to increase the size of the output in general)	91 94 <code>if_return</code> (default: <code>true</code>) – optimizations for <code>if/return</code> and <code>if/continue</code> <code>inline</code> (default: <code>true</code>) – inline calls to function with <code>simple/return</code> statement: <code>false</code> – same as <code>0</code> <code>0</code> – disabled inlining <code>1</code> – inline simple functions <code>2</code> – inline functions with arguments <code>3</code> – inline functions with arguments and variables <code>true</code> – same as <code>3</code>	92 93 <code>join_vars</code> (default: <code>true</code>) – join consecutive var, let and const statements <code>keep_classnames</code> (default: <code>false</code>) – Pass <code>true</code> to prevent the compressor from discarding class names. Pass a regular expression to only keep class names matching that regex. See also: the <code>keep_classnames</code> mangle option. <code>keep_fargs</code> (default: <code>true</code>) – Prevents the compressor from discarding unused function arguments. You need this for code which relies on <code>Function.length</code>.
96 those functions do not produce side effects. DANGER: will not check if the name is redefined in scope. An example case here, for <code>instance var q = Math.floor(a/b)</code>. If <code>drop it</code>, but will still keep the <code>Math.floor(a/b)</code>, not knowing what it does. You can pass <code>pure_funcs: ['Math.floor']</code> to let it know that this function won't produce any side effect, in which case the whole statement would get discarded. The current	95 the parens that the code generator would insert. <code>passes</code> (default: 1) – The maximum number of times to run <code>compress</code>. In some cases more than one pass leads to further compressed code. Keep in mind more passes will take more time. <code>properties</code> (default: <code>true</code>) – rewrite property access using the dot notation, for example <code>foo["bar"] -> foo.bar</code> <code>pure_funcs</code> (default: <code>null</code>) – You can pass an array of names and Terser will assume that	94 <code>lhs_constants</code> (default: <code>true</code>) – Moves constant values to the left-hand side of binary nodes. <code>foo == 42 -> 42 == foo</code> <code>loops</code> (default: <code>true</code>) – optimizations for <code>do, while</code> and for loops when we can statically determine the condition. <code>module</code> (default <code>false</code>) – Pass <code>true</code> when compressing an ES6 module. Strict mode is implied and the <code>toplevel</code> option as well. <code>negate_iife</code> (default: <code>true</code>) – negate "Immediately-Called Function Expressions" where the return value is discarded, to avoid	93 <code>keep_fnames</code> (default: <code>false</code>) – Pass <code>true</code> to prevent the compressor from discarding function names. Pass a regular expression to only keep function names matching that regex. Useful for code relying on <code>function.prototype.name</code>. See also: the <code>keep_fnames</code> mangle option. <code>keep_infinity</code> (default: <code>false</code>) – Pass <code>true</code> to prevent <code>Infinity</code> from being compressed into <code>1/0</code>, which may cause performance issues on Chrome.

- implementation adds some overhead (compression will be slower). - pure_getters (default: "strict") – If you pass true for this, Terser will assume that object property access (e.g. foo.bar or foo["bar"]) doesn't have any side effects. Specify "strict" to treat foo.bar as side-effect-free only when foo is certain to not throw, i.e. not null or undefined. - pure_new (default: false) – Set to true to assume new X() never has side effects.	- reduce_vars (default: true) – Improve optimization on variables assigned with and used as constant values. - reduce_funcs (default: true) – Inline single-use functions when possible. Depends on reduce_vars being enabled. Disabling this option sometimes improves performance of the output code. - sequences (default: true) – join consecutive simple statements using the comma operator. May be set to a positive integer to specify the maximum number of	consecutive comma sequences that will be generated. If this option is set to true then the default sequences limit is 200. Set option to false or 0 to disable. The smallest sequences length is 2. A sequences value of 1 is grandfathered to be equivalent to true and as such means 200. On rare occasions the default sequences limit leads to very slow compress times in which case a value of 20 or less is recommended.	- side_effects (default: true) – Remove expressions which have no side effects and whose results aren't used. - switches (default: true) – de-duplicate and remove unreachable switch branches - toplevel (default: false) – drop unreferenced functions ("funcs") and/or variables ("vars") in the top level scope (false by default, true to drop both unreferenced functions and variables) - top_retain (default: null) – prevent specific topLevel functions and variables from
97	98	99	100
101	101	102	101
* x, which may give imprecise floating point results. - unsafe_symbols (default: false) – removes keys from native Symbol declarations, e.g Symbol("kDog") becomes Symbol(). - unsafe_methods (default: false) – Converts function(m){} to { m: {} }(.ecma must be set to 6 or greater to enable this transform. If unsafe_methods is a RegExp then key/value pairs with keys matching the RegExp will be converted to concise methods.	get, or valueOf. This could cause change in execution order after operands in the comparison are switching. Or if one of two operands is NaN, the result is always false. Compression only works if both comparisons and unsafe_comps are both set to true. - unsafe_function (default: false) – compress and mangle function(args, literals. - unsafe_math (default: false) – optimize numerical expressions like 2 * x * 3 into 6	arrow functions if the function body does not reference this. Note: It is not always safe to perform this conversion if code relies on the functions lack. This transform requires that the ecma compress option is set to 2015 or greater. unsafe_comps (default: false) – Reverse < and >= to > and <=> to allow improved compression. This might be unsafe when an at least one of two operands is an object with computed values due the use of methods like	unused removal (can be array, comma-separated, RegExp or function. Implies topLevel) - typedefs (default: true) – Transforms typeof foo == "undefined" into foo === void 0. Note: recommend to set this value to false for IE10 and earlier versions due to known issues. - unsafe (default: false) – apply "unsafe" transformations (see details). - unsafe_arrows (default: false) – Convert ES5 style anonymous function expressions to
Note: if enabled there is a risk of getting a "<method name> is not a constructor" TypeError should any code try to new the former function. - unsafe_proto (default: false) – optimize expressions like Array.prototype.slice.call(a) into [].slice.call(a) - unsafe_regexp (default: false) – enable substitutions of variables with RegExp values the same way as if they are constants.	- unsafe_undefined (default: false) – substitute void 0 if there is a variable named undefined in scope (variable name will be mangled, typically reduced to a single character) - unused (default: true) – drop unreferenced functions and variables (simple direct variable assignments do not count as references unless set to "keep_assign")	Mangle options - eval (default false) – Pass true to mangle names visible in scopes where eval or with are used. - keep_classnames (default false) – Pass true to not mangle class names. Pass a regular expression to only keep class names matching that regex. See also: the keep_classnames compress option. - keep_fnames (default false) – Pass true to not mangle function names. Pass a regular	expression to only keep function names matching that regex. Useful for code relying on Function.prototype.name. See also: the keep_fnames compress option . - module (default false) – Pass true an ES6 modules, where the topLevel scope is not the global scope. Implies topLevel and assumes input code is strict mode JS. - nth_identifier (default: an internal mangler that weights based on character frequency analysis) – Pass an object with a get(n) function that converts an ordinal into
105	106	107	108
112	111	110	109
Mangle properties options - builtins (default: false) — Use true to allow the mangling of builtin DOM properties. Not recommended to override this setting. - debug (default: false) — Mangle names with the original name still present. Pass an	<pre>var code = fs.readFileSync("test.js", "utf8"); await minify(code).code; // 'function funcName(a,n){var globalVar;' await minify(code, { mangle: { reserved: ['firstLongName'] } }).code; // 'function funcName(firstLongName,a){var globalVar;' function funcName(firstLongName, anotherLongName) { var myVariable = firstLongName + anotherLongName; }</pre>	safari10 (default false) – Pass true to work around the Safari 10 loop iterator bug format option. Examples: also: the safari10 format option.	the nth most favored (usually shortest) identifier. Optionally also provide reset(), sort(), and consider(chars, delta) to use character frequency analysis of the source code. - reserved (default []) – Pass an array of identifiers that should be excluded from mangling. Example: ["foo", "bar"]. - toplevel (default false) – Pass true to mangle names declared in the top level scope.

empty string "" to enable, or a non-empty string to set the debug suffix. - keep_quoted (default: false) — How quoting properties ({ "prop": ... } and obj["prop"]) controls what gets mangled. "strict" (recommended) – obj.prop is mangled. - false – obj["prop"] is mangled. - true – obj.prop is mangled unless there is obj["prop"] elsewhere in the code. - nth_identifier (default: an internal mangler that weights based on character	frequency analysis) – Pass an object with a get(n) function that converts an ordinal into the nth most favored (usually shortest) identifier. Optionally also provide reset(), sort(), and consider(chars, delta) to use character frequency analysis of the source code. - regex (default: null) — Pass a RegExp literal or pattern string to only mangle property matching the regular expression. - reserved (default: []) — Do not mangle property names listed in the reserved array.	undeclared (default: false) - Mangle those names when they are accessed as properties of known top level variables but their declarations are never found in input code. May be useful when only minifying parts of a project. See #397 for more details.	Format options These options control the format of Terser’s output code. Previously known as “output options”. - ascii_only (default false) – escape Unicode characters in strings and regexps (affects directives with non-ascii characters becoming invalid) - beautify (default false) – (DEPRECATED) whether to beautify the output. When using
113	114	115	116
120 - max_line_len (default false) – maximum line length (for minified code) - preamble (default null) – when passed it must be a string and it will be prepended to the output literally. The source map will adjust for this text. Can be used to insert a comment containing licensing information, for example. - quote_keys (default false) – pass true to quote all keys in literal objects	119 - indent_start (default 0) – prefix all lines by that many spaces - inline_script (default true) – escape HTML comments and the slash in occurrences of </script> in strings - keep_numbers (default false) – keep number literals as it was in original code (disables optimizations like converting 1000000 into 1e6) - keep_quoted_props (default false) – when turned on, prevents stripping quotes from property names in object literals.	118 - output, a regular expression string (e.g. / \i/) or a function. - ecma (default 5) – set desired EcmaScript standard version for output. Set ecma to 2015 or greater to emit shorthand object properties - i.e.: {a} instead of {a: a}. The ecma option will only change the output in direct control of the beautifier. Non-compatible features in your input will still be output as is. For example: an ecma setting of 5 will not convert modern code to ES5. - indent_level (default 4)	117 - braces (default false) – always insert braces in if, for, do, while or with statements, even if their body is a single statement. - comments (default "some") – by default it keeps SDoc-style comments that contain "@license", "@copyright", "@preserve" or start with '!', pass true or "all" to preserve all comments, false to omit comments in the the legacy -b CLI flag, this is set to true by default.
- quote_style (default 0) – preferred quote style for strings (affects quoted property names and directives as well): 0 – prefers double quotes, switches to single quotes when there are more double quotes in the string itself. 0 is best for gzip size. 1 – always use single quotes 2 – always use double quotes 3 – always use the original quotes - preserve_annotations – (default false) – Preserve Terser annotations in the output.	- safari10 (default false) – set this option to true to work around the Safari 10/11 await bug. See also: the safari10 mangle option. - semicolons (default true) – separate statements with semicolons. If you pass false then whenever possible we will use a newline instead of a semicolon, leading to more readable output of minified code (size before gzip could be smaller; size after gzip insignificantly larger).	- shebang (default true) – preserve shebang #! in preamble (bash scripts) - spidermonkey (default false) – produce a Spidermonkey (Mozilla) AST - webkit (default false) – enable workarounds for WebKit bugs. PhantomJS users should set this option to true. - wrap_iife (default false) – pass true to wrap immediately invoked function expressions. See #640 for more details. - wrap_func_args (default false) – pass true in order to wrap function expressions	that are passed as arguments, in parenthesis. See OptimizeJS for more details.
121	122	123	124
128 The unsafe compress option It enables some transformations that <i>might</i> break code logic in certain contrived cases, but should be fine for most code. It assumes that standard built-in ECMAScript functions and classes have not been altered or replaced. You might want to try it on your own code; it should reduce the minified size. needs to be kept in the output) are comments attached to toplevel nodes.	127 // this function is never called return something(); } Even though it has "@preserve", the comment will be lost because the inner function g (which is the AST node to which the comment is attached to) is discarded by the compressor as not referenced. The safest comments where to place copyright information (or other info that	126 example: <pre>function f() { /**@preserve Foo Bar */ function g() { // this function is never called return something(); } }</pre> Note, however, that there might be situations where comments are lost. For example like /*! Copyright Notice */. comments that match this regexp. For a valid JavaScript regexp to keep only comments all to keep all the comments, or	125 Miscellaneous Keeping copyright notices or other comments You can pass --comments to retain certain comments in the output. By default it will keep comments starting with "!" and JSDoc-style comments that contain "@preserve", "@copyright", "@license" or "@cc_on" (conditional compilation for IE). You can pass

[illegible]

[illegible]



161

- 38elements

Contributors

Code Contributors

This project exists thanks to all the people who contribute. [\[Contribute\]](#).



163

Financial Contributors

Become a financial contributor and help us sustain our community. [\[Contribute\]](#)

Individuals



164



891



162



991



Support this project with your organization. Your logo will show up here with a link to your website. [\[Contribute\]](#)

Organizations



169



170



171



172

176

175

174

173

